Set_A_Lab9 - Results

Attempt 1 of 1

Written Mar 25, 2026 9:04 PM - Mar 25, 2026 9:15 PM

Attempt Feedback

30% penalty
17/18
Submitted Late: 1 hour past the due date
Absent Mar 25

Attempt Score	**11.9 / 18 - 66.11 %**
Overall Grade (Highest Attempt)	**11.9 / 18 - 66.11 %**

Question 1 — 1 / 1 point

Use the online Linux (https://cocalc.com/) and run the provided MPI code (Q1.c) with two processes.

Q1.c

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, size;
    int a[10] = {1,2,3,4,5,6,7,8,9,10};
    int local_sum = 0, global_sum = 0;
    int i;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2)
    {
        if (rank == 0)
            printf("Please run the program with exactly 2 processes.\n");
        MPI_Finalize();
        return 0;
    }
    if (rank == 0)
    {
        for (i = 0; i < 5; i++)
            local_sum += a[i];
    }
    else if (rank == 1)
    {
        for (i = 5; i < 10; i++)
            local_sum += a[i];
    }
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rank == 0)
        printf("Sum of array elements = %d\n", global_sum);

    MPI_Finalize();
    return 0;
}
```

```
mpicc Q1.c -o Q1
mpirun -np  2 Q1
```

Observe and record the output. Insert a screenshot of the output.

```
~$ mpicc Q1.c -o Q1
~$ mpirun -np 2 Q1
Sum of array elements = 55
```

Screenshot 2025-09-27 151234.png (14.13 KB)

Screenshot 2026-03-25 210726.png (2.2 KB)

The correct answer is not displayed for Written Response type questions.

Question 2

1 / 1 point

Referring to your results from Question 1, how many elements of the array does each process sum?

- ✔ (selected) 5
- 2
- Depends on the input
- 10

Question 3

1 / 1 point

Referring to your results from Question 1, which MPI function is used to combine partial sums from all processes?

- MPI_Gather
- MPI_Send
- MPI_Bcast
- ✔ (selected) MPI_Reduce

Question 4

1 / 1 point

Referring to your results from Question 1, why does the given MPI program not suffer from a race condition when computing the final sum?

- ○ Because the array is declared locally inside main()
- ✓ ◉ Because each process works on a separate portion of the array and uses MPI_Reduce
- ○ Because both processes access the array a at the same time
- ○ Because MPI programs cannot have race conditions

Question 5

1 / 1 point

Use https://www.onlinegdb.com/online_c_compiler, run the following code and answer or complete the following questions.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ARRAY_SIZE 16

int array[ARRAY_SIZE];
int partial_sums[NUM_THREADS];
pthread_barrier_t barrier;

void* worker(void* arg) {
    int id = *(int*)arg;
    int start = id * (ARRAY_SIZE / NUM_THREADS);
    int end = start + (ARRAY_SIZE / NUM_THREADS);

    // Each thread computes sum of its chunk
    partial_sums[id] = 0;
    for (int i = start; i < end; i++) {
        partial_sums[id] += array[i];
    }
    printf("Thread %d: partial sum = %d\n", id, partial_sums[id]);

    // pthread_barrier_wait(&barrier);

    if (id == 0) {
        int total = 0;
        for (int i = 0; i < NUM_THREADS; i++)
            total += partial_sums[i];
        printf("Thread %d: total sum = %d\n", id, total);
    }
    free(arg);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    // Initialize array with numbers 1..16
    for (int i = 0; i < ARRAY_SIZE; i++) array[i] = i + 1;

    // Initialize barrier
    pthread_barrier_init(&barrier, NULL, NUM_THREADS);

    for (int i = 0; i < NUM_THREADS; i++) {
        int* id = malloc(sizeof(int));
        *id = i;
        pthread_create(&threads[i], NULL, worker, id);
    }

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);
```

```
53
54      pthread_barrier_destroy(&barrier);
55      return 0;
56  }
57
```

Capture and insert a screenshot of the program's output.

```
Thread 1: partial sum = 26
Thread 0: partial sum = 10
Thread 0: total sum = 136
Thread 2: partial sum = 42
Thread 3: partial sum = 58
```

Screenshot 2026-03-25 210950.png (3.23 KB)

The correct answer is not displayed for Written Response type questions.

Question 6 — 0 / 1 point

Referring to your results from Question 5, execute your code multiple times. Does the value of the total variable change each time?

- ✖ ◉ No
- ➡ ○ Yes

Question 7 — 1 / 1 point

Referring to your results from Question 5 uncomment line 24 and run the code several times. Does the value of the total variable still change?

- ○ Yes
- ✔ ◉ No

Question 8 — 1 / 1 point

Shared memory IPC requires explicit synchronization mechanisms such as semaphores or mutexes.

- ✔ (●) True
- ( ) False

Question 9 1 / 1 point

Unnamed pipes are typically used between related processes such as parent and child.

- ✔ (●) True
- ( ) False

Question 10 1 / 1 point

Race conditions cannot occur in shared memory IPC.

- ( ) True
- ✔ (●) False

Question 11 1 / 1 point

Which IPC mechanism is most suitable for **communication between processes on different machines**?

- ( ) Pipes
- ( ) Signals
- ✔ (●) Sockets
- ( ) Shared memory

Question 12 1 / 1 point

Which IPC mechanism works using **file descriptors**?

- ✔ (selected) Pipes
- Signals
- Shared memory
- Semaphores

Question 13

1 / 1 point

Two Processes Communicating Using Named Semaphore

Use https://www.onlinegdb.com and run your code. Insert a screenshot of the output.

JavaScript

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <semaphore.h>
#include <sys/wait.h>

#define SEM_NAME "/my_named_semaphore"

int main() {

    sem_t *sem;
    sem = sem_open(SEM_NAME, O_CREAT, 0644, 1);
    if (sem == SEM_FAILED) {
        perror("sem_open");
        exit(1);
    }
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(1);
    }
```

```
    if (pid == 0) {
        sleep(1);
        sem_wait(sem);
        FILE *fp = fopen("shared.txt", "r");
        if (fp != NULL) {
            char buffer[100];
            fgets(buffer, sizeof(buffer), fp);
            printf("Child Process Read: %s\n", buffer);
            fclose(fp);
        }
        sem_post(sem);
    }
    else {
        sem_wait(sem);
        FILE *fp = fopen("shared.txt", "w");
        if (fp != NULL) {
            fprintf(fp, "Hello from Parent Process\n");
            fclose(fp);
            printf("Parent Process Wrote Data\n");
        }
        sem_post(sem);   // unlock
        wait(NULL);
    }
    sem_close(sem);
    sem_unlink(SEM_NAME);

    return 0;
}
```

```
Parent Process Wrote Data
Child Process Read: Hello from Parent Process
```

Screenshot 2026-03-25 211307.png (2.4 KB)

The correct answer is not displayed for Written Response type questions.

Question 14

1 / 1 point

Referring to your results from Question 13, why is sem_open() used instead of sem_init() in this program?

- ○ Because threads are used

- [x] Because the semaphore must be shared between processes
- [ ] Because the semaphore is temporary
- [ ] Because it is faster

Question 15 1 / 1 point

Referring to your results from Question 13, what is the purpose of `sem_wait(sem)` in the program?

- [ ] Create a new process
- [x] Acquire the semaphore and enter critical section
- [ ] Increment semaphore value
- [ ] Destroy the semaphore

Question 16 1 / 1 point

Referring to your results from Question 13, if the semaphore initial value is changed from 1 to 0, what effect does it have?

- [ ] Both processes may enter the critical section simultaneously
- [ ] The semaphore becomes invalid
- [x] No process can access the critical section
- [ ] Only the parent process runs

Question 17

1 / 1 point

Referring to your results from Question 13, why is sem_unlink(SEM_NAME) called at the end of the program?

- ✔ (selected) To remove the named semaphore from the system
- To close the semaphore for the current process
- To increase semaphore value
- To reset semaphore value

Question 18

1 / 1 point

Referring to your results from Question 13, what could happen if both processes remove the semaphore operations?

- Program will not compile
- Both processes terminate immediately
- File cannot be opened
- ✔ (selected) Race condition may occur when accessing shared.txt

Done